SOC ANALYST PORTFOLIO CASE STUDY

Detecting a Web Application SQL-Injection Attempt

Baseline comparison and Wazuh SIEM investigation in a virtualized home lab

RYAN BRYANT | SOC / CYBERSECURITY ANALYST CANDIDATE | AUGUST 2026

OUTCOME Integrated Apache access telemetry with Wazuh, established a benign baseline, generated a controlled SQL-injection-style request, and validated a level 7 alert mapped to MITRE ATT&CK T1190.

Figure 1. Home-lab architecture and Apache telemetry flow.

Portfolio note: This case study documents an authorized simulation against a local Apache server. No production application, database, third-party account, or unauthorized target was involved. AI assistance is disclosed within this report.

1. Executive Summary

I expanded my Wazuh home lab from authentication monitoring into web-application telemetry and detection. I installed Apache HTTP Server on the Ubuntu endpoint, configured rsyslog imfile collection for the Apache access log, and forwarded those records to Wazuh through the existing TCP syslog channel.

I first generated a benign request to establish a comparison point. I then submitted one harmless SQL-injection-style URL to a nonexistent local page. Both requests returned HTTP 404, but Wazuh distinguished the malicious query syntax and raised rule 31103 at level 7, classifying it as an SQL injection attempt and mapping it to MITRE ATT&CK T1190, Exploit Public-Facing Application.

Project dimension	Implementation
Role performed	Lab architect, log engineer, detection tester, and Tier 1 analyst
Telemetry source	Apache 2.4 access log on linux-webserver01
Transport	rsyslog imfile → TCP syslog 514 → Wazuh 10.10.10.3
Baseline	Benign missing-page request; HTTP 404; Wazuh level 5
Detection	SQL injection attempt; Wazuh rule 31103; level 7
Framework mapping	MITRE ATT&CK T1190 — Exploit Public-Facing Application
Disposition	True positive, expected/benign due to controlled simulation

2. Objectives

- Add application-layer telemetry to the existing SOC lab.
- Validate Apache access-log ingestion through rsyslog and Wazuh.
- Compare normal web traffic with a malicious-looking request.
- Investigate the decoded HTTP fields and Wazuh rule metadata.
- Document severity, MITRE mapping, scope, and response recommendations.

3. Lab Architecture and Configuration

Component	Configuration	Purpose
Wazuh SIEM	10.10.10.3; Wazuh 4.14.7	Receives, decodes, correlates, and displays alerts
Web endpoint	linux-webserver01; 10.10.10.4; Ubuntu 24.04.4 LTS	Hosts Apache and forwards access records
Web service	Apache 2.4.58; TCP port 80	Creates standard access-log telemetry
Log collector	rsyslog imfile; tag apache-access	Reads /var/log/apache2/access.log
Transport	TCP syslog port 514	Forwards application events to Wazuh
Test source	127.0.0.1 on the Ubuntu server	Keeps all requests inside the authorized endpoint

4. Implementation Workflow

01 WEB SERVICE DEPLOYMENT
Installed Apache from the signed Ubuntu repositories, confirmed the service was active, and received HTTP/1.1 200 OK from the local default page.

02 ACCESS-LOG COLLECTION
Loaded the rsyslog imfile input module and configured it to monitor /var/log/apache2/access.log with the tag apache-access.

03 CONFIGURATION VALIDATION
Ran rsyslogd -N1, received a successful validation result, restarted rsyslog, and confirmed the service remained active.

04 BASELINE GENERATION
Requested /soc-baseline-test with user-agent SOC-Lab-Baseline. Apache returned 404 and Wazuh indexed one level 5 event.

05 CONTROLLED SQLI SIMULATION
Requested a nonexistent local PHP path containing URL-encoded union select and from terms. Apache returned 404; no application or database query executed.

06 ALERT INVESTIGATION
Filtered Discover using the unique SOC-Lab-SQLi-Test marker, expanded the record, and reviewed the decoder, HTTP fields, rule metadata, compliance tags, and MITRE mapping.

5. Detection Engineering Details

Apache imfile collection

```
module(load="imfile")
input(type="imfile" File="/var/log/apache2/access.log"
    Tag="apache-access:" Severity="info" Facility="local6")
```

Controlled request

```
GET /index.php?id=1%20union%20select%20username,password%20from%20users-- HTTP/1.1
```

SAFETY BOUNDARY The request targeted 127.0.0.1 and a nonexistent page. It produced a log pattern for detection testing but did not execute SQL, access a database, or exploit an application.

6. Evidence: Benign Baseline

The baseline established that the pipeline could ingest ordinary Apache activity before testing malicious syntax. The request used a unique user-agent and a nonexistent path, producing one expected 404 event at rule level 5.

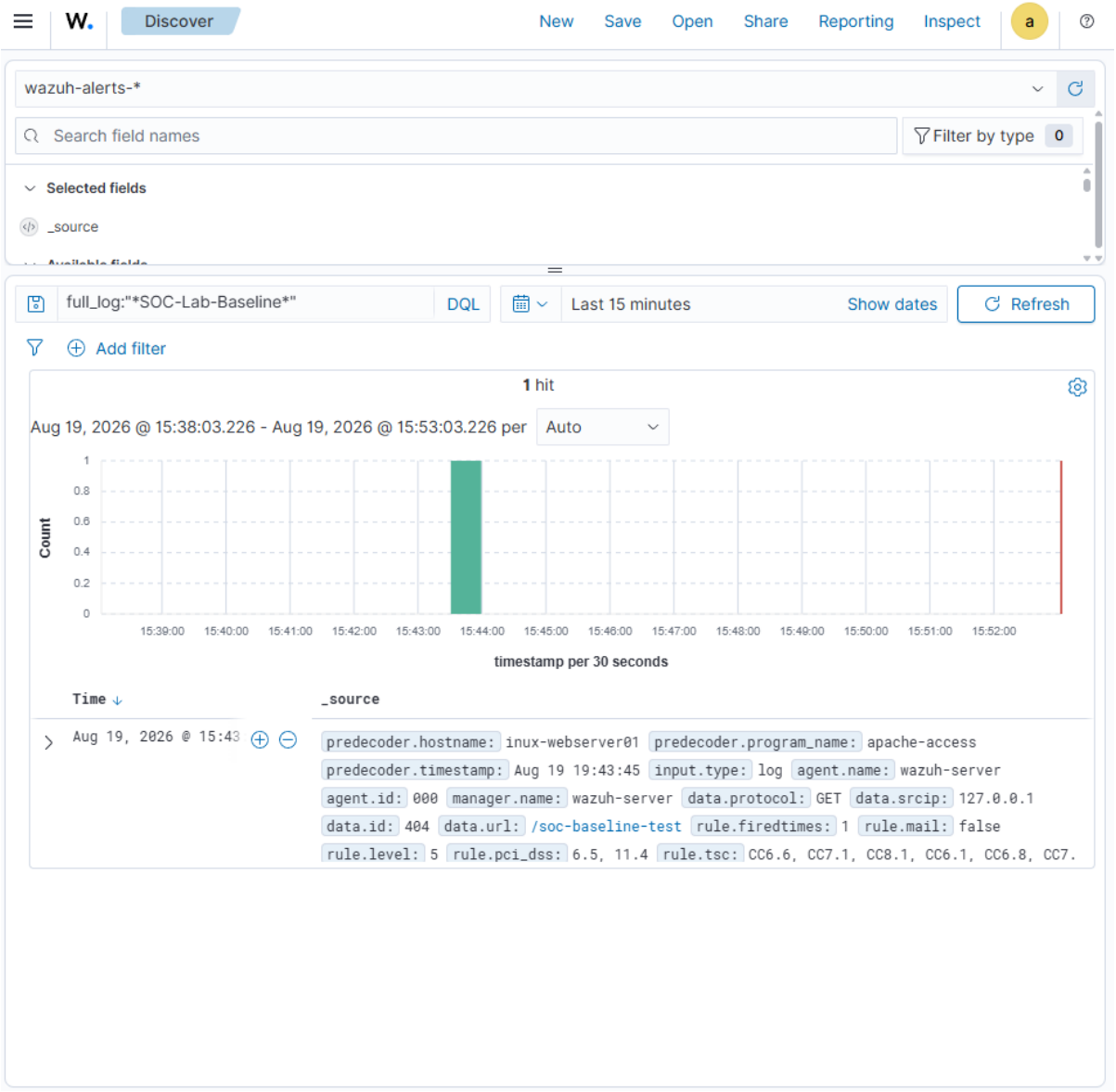


Figure 2. Baseline web request indexed in Wazuh before the SQL-injection simulation.

Baseline field	Observed value
Program	apache-access
Method	GET
URL	/soc-baseline-test
Source IP	127.0.0.1
HTTP response	404 Not Found
Alert severity	Wazuh rule level 5

7. Evidence: Controlled Request

The terminal evidence records the exact local request and the resulting Apache access-log entry. The response remained 404, confirming that the nonexistent application path did not execute.

Figure 3. Local SQL-injection-style request and corresponding Apache access record.

ANALYST OBSERVATION The request outcome alone did not indicate compromise. Detection depended on the structure of the URL, not on a successful HTTP response.

8. Evidence: Event-Level Investigation

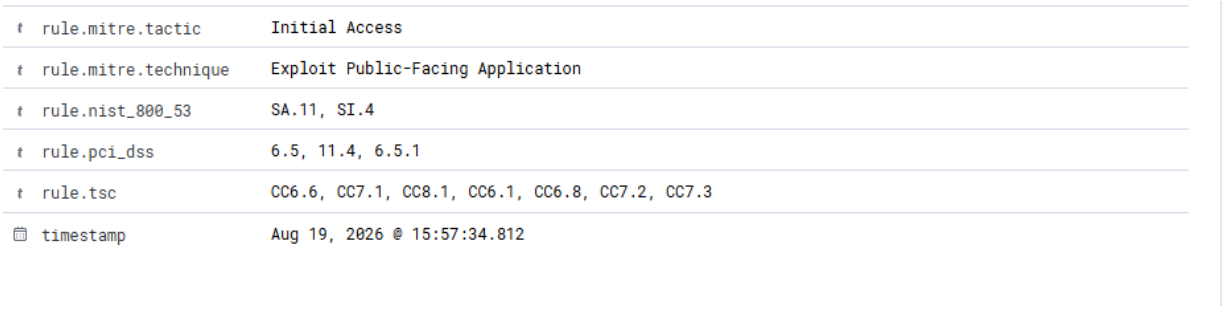
Wazuh decoded the forwarded record as a web access log and extracted the source IP, HTTP method, response code, and complete URL. Rule 31103 recognized the SQL-injection pattern and elevated the event to level 7.

t	_index	wazuh-alerts-4.x-2026.08.19
t	agent.id	000
t	agent.name	wazuh-server
t	data.id	404
t	data.protocol	GET
t	data.srcip	127.0.0.1
t	data.url	/index.php?id=1%20union%20select%20username,password%20from%20users--
t	decoder.name	web-accesslog
t	full_log	Aug 19 19:57:34 inux-webserver01 apache-access: 127.0.0.1 - - [19/Aug/2026:19:57:34 +0000] "GET /index.php?id=1%20union%20select%20username,password%20from%20users-- HTTP/1.1" 404 432 "-" "SOC-Lab-SQLi-Test"
t	id	1787169454.73462
t	input.type	log
t	location	10.10.10.4
t	manager.name	wazuh-server
t	predecoder.hostname	inux-webserver01
t	predecoder.program_name	apache-access
t	predecoder.timestamp	Aug 19 19:57:34
t	rule.description	SQL injection attempt.
#	rule.firedtimes	1
t	rule.gdpr	IV_35.7.d
t	rule.groups	web, accesslog, attack, sql_injection
t	rule.id	31103
#	rule.level	7
◕	rule.mail	false
t	rule.mitre.id	T1190
t	rule.mitre.tactic	Initial Access
t	rule.mitre.technique	Exploit Public-Facing Application
t	rule.nist_800_53	SA.11, SI.4

Figure 4. Expanded Wazuh SQL-injection alert and decoded HTTP fields.

Field	Observed value
Endpoint / location	linux-webserver01 / 10.10.10.4
Decoder	web-accesslog
Program	apache-access
Source IP	127.0.0.1
HTTP method / result	GET / 404
Rule	31103 — SQL injection attempt
Severity	Level 7
Groups	web, accesslog, attack, sql_injection

9. MITRE ATT&CK and Compliance Context



t	rule.mitre.tactic	Initial Access
t	rule.mitre.technique	Exploit Public-Facing Application
t	rule.nist_800_53	SA.11, SI.4
t	rule.pci_dss	6.5, 11.4, 6.5.1
t	rule.tsc	CC6.6, CC7.1, CC8.1, CC6.1, CC6.8, CC7.2, CC7.3
🗓	timestamp	Aug 19, 2026 @ 15:57:34.812

Figure 5. MITRE ATT&CK and compliance metadata associated with the alert.

Classification	Value
MITRE tactic	Initial Access
MITRE technique	T1190 — Exploit Public-Facing Application
NIST SP 800-53	SA.11; SI.4
PCI DSS	6.5; 11.4; 6.5.1
TSC	CC6.6; CC7.1; CC8.1; CC6.1; CC6.8; CC7.2; CC7.3

The MITRE mapping describes the behavior category detected in the request. It does not prove that exploitation succeeded. In this lab, the 404 response, nonexistent application, and known simulation context ruled out compromise.

10. Analyst Triage and Disposition

Question	Assessment
What happened?	A local HTTP request contained URL-encoded SQL union/select/from syntax.
What was targeted?	A nonexistent /index.php path on the Ubuntu Apache server.
Was it successful?	No. Apache returned 404 and no database-backed application existed.
Why did Wazuh alert?	Rule 31103 matched SQL-injection indicators in the decoded URL.
What was the scope?	One controlled attack event from 127.0.0.1.
How should it be classified?	True positive detection; expected benign activity in an authorized lab.

FINAL DISPOSITION TRUE POSITIVE — EXPECTED LAB ACTIVITY. Wazuh correctly identified the SQL-injection pattern. No exploitation, unauthorized access, or data exposure occurred.

11. Recommended Response in a Real Environment

- Validate the client IP, requested URI, HTTP method, user-agent, response code, and affected application owner.
- Search for repeated SQL-injection patterns and successful 2xx responses from the same source.
- Review web-server, WAF, application, database, authentication, and endpoint telemetry for correlated activity.
- Block or rate-limit a confirmed hostile source and apply WAF rules where appropriate.
- Assess the application for parameterized queries, input validation, least-privilege database access, and secure error handling.
- Preserve evidence, document scope and timeline, and escalate if execution, data access, or persistence is observed.

12. Troubleshooting and Engineering Judgment

Challenge	Resolution and lesson
Application logs were not automatically forwarded	Added rsyslog imfile monitoring for Apache access.log. Lesson: verify the telemetry source at the application layer.
Needed to prove the pipeline before attack testing	Created a unique benign baseline event and located one corresponding Wazuh record.
A 404 could be mistaken for a harmless event	Compared rule metadata and URL content; Wazuh elevated the SQLi pattern despite the same response class.
Agentless visibility tradeoff	Documented that syslog supplied log-based detection but not endpoint file-integrity or process telemetry.

13. Assistance and Source Transparency

This project was completed by Ryan Bryant in his personal home lab with assistance from OpenAI ChatGPT (referred to conversationally as “Sarah”). ChatGPT helped interpret results, suggest safe test and validation steps, locate official documentation, and edit this portfolio narrative. Ryan personally installed and configured Apache and rsyslog, entered the commands, generated the requests, investigated the Wazuh records, captured the screenshots, and verified the findings. AI suggestions were evaluated against observed system output and the cited sources; ChatGPT did not independently access or operate the lab computer.

14. Skills Demonstrated

- Web-server deployment and validation
- Application-log collection with rsyslog imfile
- Centralized TCP syslog engineering
- Baseline-versus-attack comparison
- HTTP access-log analysis
- SIEM alert triage and disposition
- MITRE ATT&CK T1190 mapping
- Evidence-based incident documentation
- Ethical testing and scope control

15. Employer-Facing Project Summary

RESUME BULLET Integrated Apache access telemetry with Wazuh via rsyslog, established a benign baseline, generated and investigated a controlled SQL-injection-style request, and validated a level 7 rule 31103 alert mapped to MITRE ATT&CK T1190.

Interview talking points

- How I proved the telemetry pipeline before testing detection logic.
- Why HTTP status alone cannot determine whether a request is malicious.
- How Wazuh extracted fields and matched SQL-injection indicators.
- How I distinguished a true detection from a successful compromise.
- What additional WAF, database, and endpoint evidence I would collect in production.

16. Next Iterations

Future lab work can add a controlled directory-traversal event, compare single-event and correlation thresholds, deploy the Wazuh endpoint agent when the official package path is available, and test how WAF or active-response controls change detection and containment evidence.

17. References and Validation Sources

These sources were used to validate the collection method, Apache log structure, Wazuh detection behavior, and AI-assistance disclosure. Direct system output and the supplied screenshots remain the primary evidence for this case study.

OpenAI key concepts for GPT text-generation assistance
Wazuh syslog receiver configuration
Wazuh Linux log collection using rsyslog
Wazuh log data analysis pipeline
Wazuh web-access detection rules
Apache HTTP Server access-log documentation
Ubuntu rsyslog forwarding syntax
MITRE ATT&CK T1190